

startTests.bat – Quick User Manual

[1. Test Suite ausführen](#)

[Syntax](#)

[1.1. Einen bestimmten TestCase ausführen](#)

[1.2. TestCase und Priority angeben](#)

[2. Konfiguration für die Testausführung](#)

[Environment-Konfiguration](#)

[User-Konfiguration](#)

[3. Template instanziiieren](#)

[3.1. Syntax](#)

[3.2. Was passiert bei instantiate?](#)

[3.3. Ablauf der Instanziierung](#)

[3.4. Warum gibt es Template und Instanz getrennt?](#)

[3.5. Beispiel aus der Tosca-Migration](#)

[3.6. Instanziierten Test ausführen](#)

[3.7. Empfohlener Workflow für Template-Tests](#)

[Schritt 1 – Template und CSV prüfen](#)

[Schritt 2 – Instanziiieren](#)

[Schritt 3 – Generiertes YAML validieren](#)

[Schritt 4 – Test ausführen](#)

[4. YAML validieren](#)

[Syntax](#)

[Erfolgreiche Validierung](#)

[Fehlgeschlagene Validierung](#)

[Empfehlung](#)

[5. AGATE DSL anzeigen](#)

[Alle verfügbaren DSL-Typen](#)

[Einen bestimmten DSL-Typ anzeigen](#)

[6. Typischer Workflow für einen normalen Test](#)

[7. Typischer Workflow für einen Template-Test](#)

[8. Quick Reference](#)

[Test ausführen](#)

[Einzelnen TestCase ausführen](#)

[Template instanziiieren](#)

[YAML validieren](#)

[DSL-Typen anzeigen](#)

[9. Merksatz](#)

`startTests.bat` ist der zentrale Kommandozeilen-Einstiegspunkt für den AGATE Server.

Über dasselbe Script können vier typische Aufgaben ausgeführt werden:

```
1 | 1. Test Suite ausführen
2 | 2. Template + Testdaten instanziiieren
3 | 3. YAML validieren
4 | 4. AGATE DSL anzeigen
```

1. Test Suite ausführen

Syntax

```
1 startTests.bat <user> <instance> <app> <testSuite> [testCase]
2 [priority]
```

Beispiel:

```
1 startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
2
```

Bedeutung:

Parameter	Beispiel	Beschreibung
user	Testuser	Benutzer, unter dem der Test ausgeführt wird
instance	ACME	Testumgebung / Konfigurationsinstanz
app	TOSCATOAGATE	AGATE Application
testSuite	Instance_AGATE_Demo.yaml	Auszuführende Test Suite
testCase	optional	Nur einen bestimmten TestCase ausführen
priority	optional	Tests nach Priorität filtern

Der Aufruf:

```
1 startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

bedeutet somit:

```
1 User      = Testuser
2 Instance  = ACME
3 Application = TOSCATOAGATE
4 Test Suite = Instance_AGATE_Demo.yaml
```

1.1. Einen bestimmten TestCase ausführen

Optional kann ein einzelner TestCase angegeben werden.

Beispiel:

```
1 startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
  AGATE_Demo
```

Damit wird nur:

```
1 AGATE_Demo
```

aus der angegebenen Test Suite ausgeführt.

1.2. TestCase und Priority angeben

Zusätzlich kann die Priorität angegeben werden:

```
1 startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
  AGATE_Demo HIGH
```

Damit kann die Testausführung zusätzlich auf die gewünschte Priorität eingeschränkt werden.

2. Konfiguration für die Testausführung

Die beim Start angegebenen Werte:

```
1 user
2 instance
```

werden für die AGATE-Konfiguration verwendet.

Für:

```
1 startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

ist beispielsweise die Instance:

```
1 ACME
```

Environment-Konfiguration

In:

```
1 env/env.conf
```

können Environment-Werte definiert werden.

Beispiel:

```
1 # --- TEST INSTANCE: ACME ---
2 ACME.math.url=http://api.mathjs.org
3
```

Im Test kann dieser Wert beispielsweise verwendet werden als:

```
1 endpoint: "${E[env.math.url]}"
```

User-Konfiguration

Auch der angegebene Benutzer muss für die Instance bekannt sein.

Für:

```
1 Instance = ACME
2 User     = Testuser
```

erwartet AGATE User-Einträge mit dem Prefix:

```
1 ACME.Testuser.
```

Beispiel:

```
1 # --- USER: ACME ---
2 ACME.Testuser.ip=127.0.0.1
3 ACME.Testuser.email=testuser@acme.local
```

Eine einzelne Zeile wie:

```
1 ACME.Testuser=Mustermann
```

reicht dafür nicht aus, da AGATE nach Konfigurationseinträgen mit:

```
1 ACME.Testuser.
```

sucht.

3. Template instanziiieren

Die Instanziiierung ist insbesondere für Template-basierte Tests wichtig, beispielsweise für Tests, die aus **Tosca TestCase Design** migriert wurden.

Dabei existieren zwei getrennte Artefakte:

```
1 Template YAML
2 +
3 CSV Testdaten
4
```

Beispiel:

```
1 AGATE_Demo.yaml
2 TCD_AGATE_Demo.csv
3
```

Diese beiden Dateien sind noch nicht die endgültige ausführbare Test Suite.

Mit `instantiate` werden Template und Testdaten kombiniert und daraus konkrete AGATE TestCases erzeugt.

3.1. Syntax

```
1 startTests.bat instantiate <appName> <templateFile> <dataFile>
2
```

Für unser Beispiel:

```
1 startTests.bat instantiate TOSCATOAGATE AGATE_Demo.yaml
2 TCD_AGATE_Demo.csv
```

Dabei gilt:

Parameter	Beispiel	Bedeutung
appName	TOSCATOAGATE	Application, in deren Datenbereich das Template liegt
templateFile	AGATE_Demo.yaml	YAML Template mit der Testlogik
dataFile	TCD_AGATE_Demo.csv v	CSV mit den konkreten Testdaten

3.2. Was passiert bei `instantiate` ?

Das Template enthält die Testlogik, aber bestimmte Werte werden nicht direkt im YAML hinterlegt.

Beispiel:

```
1 - type: BUFFER
2   op: EXEC
3   name: 'x'
4   value: '{XL[x]}'
5
6 - type: BUFFER
7   op: EXEC
8   name: 'y'
9   value: '{XL[y]}'
10
11 - type: BUFFER
12   op: EXEC
13   name: 'z'
14   value: '{XL[z]}'
15
```

Die Ausdrücke:

```
1 {XL[x]}
2 {XL[y]}
3 {XL[z]}
```

verweisen auf Werte aus der CSV-Datei.

Beispielsweise kann die CSV unterschiedliche Datensätze enthalten:

```
1 x ; y ; z
2 2 ; 7 ; 3
3 22 ; 70 ; 3
```

Das Template definiert somit:

```
1 WAS getestet wird
```

und die CSV definiert:

```
1 MIT WELCHEN DATEN getestet wird
```

`instantiate` kombiniert beides.

3.3. Ablauf der Instanziierung

Der Ablauf kann vereinfacht so dargestellt werden:

```
1 AGATE_Demo.yaml
2   +
3   TCD_AGATE_Demo.csv
4     |
5     v
6 startTests.bat instantiate
7     |
8     v
9 Template wird gelesen
10    |
11    v
12 CSV-Datensätze werden gelesen
13    |
14    v
15 {XL[...] } werden mit konkreten Werten aufgelöst
16    |
17    v
18 konkrete TestCases werden erzeugt
19    |
20    v
21 ausführbare AGATE Test Suite
22
```

Beispiel:

Das Template enthält:

```
1 name: 'x'
2 value: '{XL[x]}'
3
```

und der CSV-Datensatz enthält:

```
1 x = 2
2
```

Dann erhält der konkrete TestCase für diesen Datensatz den Wert:

$$\begin{array}{c|c} 1 & x = 2 \\ 2 & \end{array}$$

Beim nächsten CSV-Datensatz könnte dagegen gelten:

1	x = 22
2	

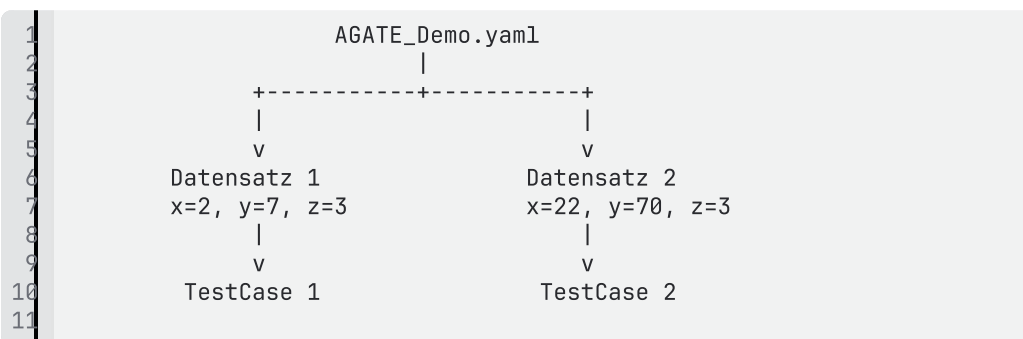
Die Testlogik bleibt identisch.

Nur die Testdaten ändern sich.

3.4. Warum gibt es Template und Instanz getrennt?

Der Vorteil ist, dass eine Testlogik nur einmal beschrieben werden muss.

Beispiel:



Ohne Templates müsste für jeden Datensatz eine eigene YAML-Testdefinition gepflegt werden.

Mit Template + CSV kann dieselbe Testlogik für viele Datenkombinationen verwendet werden.

3.5. Beispiel aus der Tosca-Migration

Nach einer Tosca-Migration können beispielsweise folgende Dateien vorhanden sein:

```
1 data/
2   └─ TOSCATOAGATE/
3       └─ template/
4           └─ AGATE_Demo.yaml
5           └─ TCD_AGATE_Demo.csv
6
```

Diese werden instanziiert mit:

```
1 startTests.bat instantiate TOSCATOAGATE AGATE_Demo.yaml
  TCD_AGATE_Demo.csv
```

Das Ergebnis ist eine konkrete AGATE Test Suite.

In unserem Beispiel kann anschließend beispielsweise:

```
1 Instance_AGATE_Demo.yaml
2
```

ausgeführt werden.

3.6. Instanziierten Test ausführen

Nach erfolgreicher Instanziierung wird der erzeugte Test genau wie jede andere AGATE Test Suite ausgeführt.

Beispiel:

```
1 startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
2
```

Der vollständige Workflow ist damit:

```
1 AGATE_Demo.yaml
2   +
3 TCD_AGATE_Demo.csv
4   |
5   v
6 instantiate
7   |
8   v
9 Instance_AGATE_Demo.yaml
10  |
11  v
12 validate
13  |
14  v
15 execute
16
```

3.7. Empfohlener Workflow für Template-Tests

Schritt 1 – Template und CSV prüfen

```
1 data/TOSCATOAGATE/template/AGATE_Demo.yaml
2 data/TOSCATOAGATE/template/TCD_AGATE_Demo.csv
3
```

Schritt 2 – Instanziiieren

```
1 startTests.bat instantiate TOSCATOAGATE AGATE_Demo.yaml
2 TCD_AGATE_Demo.csv
```

Schritt 3 – Generiertes YAML validieren

Beispiel:

```
1 startTests.bat validate data\TOSCATOAGATE\Instance_AGATE_Demo.yaml
2
```

Schritt 4 – Test ausführen

```
1 startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
2
```

Damit ist der gesamte Template-Workflow:

```
1 Template
2   +
3   Testdaten
4     |
5     v
6   INSTANTIATE
7     |
8     v
9   konkrete Test Suite
10    |
11    v
12  VALIDATE
13    |
14    v
15  EXECUTE
16
```

4. YAML validieren

Mit `validate` kann eine AGATE YAML-Datei geprüft werden, ohne den eigentlichen Test auszuführen.

Syntax

```
1 startTests.bat validate <yamlFile>
2
```

Beispiel:

```
1 startTests.bat validate data\TOSCATOAGATE\Instance_AGATE_Demo.yaml
2
```

Ein weiteres Beispiel:

```
1 startTests.bat validate data\MUHI\CheckStatus.yaml
2
```

Bei Pfaden mit Leerzeichen müssen Anführungszeichen verwendet werden:

```
1 startTests.bat validate "data\DMP21\DMP SS12 V11
2   getBetreutePatienten.yaml"
```

/

Erfolgreiche Validierung

Bei Erfolg erscheint:

```
1 =====
2   VALIDATION SUCCESSFUL
```

```
3 =====
4
```

Die Datei kann anschließend ausgeführt werden.

Fehlgeschlagene Validierung

Bei Fehlern erscheint:

```
1 =====
2          VALIDATION FAILED
3 =====
4
```

Zusätzlich liefert das Script einen Fehler-Exit-Code zurück.

Dies ist insbesondere auch für CI/CD-Pipelines nützlich.

Empfehlung

Neue oder geänderte Tests sollten vor der Ausführung grundsätzlich validiert werden:

```
1  YAML
2  |
3  v
4  validate
5  |
6  +---- Fehler ----> korrigieren
7  |
8  v
9  execute
10
```

5. AGATE DSL anzeigen

Mit `describe` kann die eingebaute AGATE DSL-Dokumentation direkt über die Kommandozeile angezeigt werden.

Alle verfügbaren DSL-Typen

```
1 startTests.bat describe
2
```

Damit erhält man eine Übersicht der verfügbaren AGATE Step Types.

Einen bestimmten DSL-Typ anzeigen

Beispiel REST:

```
1 startTests.bat describe REST
2
```

SOAP:

```
1 startTests.bat describe SOAP
2
```

Reusable / CALL:

```
1 startTests.bat describe CALL
2
```

Weitere Beispiele:

```
1 startTests.bat describe SQL
2 startTests.bat describe BUFFER
3 startTests.bat describe CMD
4 startTests.bat describe FILE
5 startTests.bat describe WAIT
6
```

`describe` ist damit die schnellste Referenz beim Schreiben eines AGATE Tests.

6. Typischer Workflow für einen normalen Test

Für eine bereits fertige Test Suite:

```
1 YAML
2 |
3 v
4 VALIDATE
5 |
6 v
7 EXECUTE
8
```

Beispiel:

```
1 startTests.bat validate data\TOSCATOAGATE\Instance_AGATE_Demo.yaml
2
```

danach:

```
1 startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
2
```

7. Typischer Workflow für einen Template-Test

Bei Template-basierten Tests kommt vorher die Instanziierung:

```
1 Template YAML
2 +
3 CSV Testdaten
4 |
5 v
6 INSTANTIATE
7 |
8 v
9 konkrete Test Suite
```

```
10 |  
11 v  
12 VALIDATE  
13 |  
14 v  
15 EXECUTE  
16
```

Konkretes Beispiel:

```
1 startTests.bat instantiate TOSCATOAGATE AGATE_Demo.yaml  
2 TCD_AGATE_Demo.csv
```

danach:

```
1 startTests.bat validate data\TOSCATOAGATE\Instance_AGATE_Demo.yaml  
2
```

und anschließend:

```
1 startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml  
2
```

8. Quick Reference

Test ausführen

```
1 startTests.bat <user> <instance> <app> <testSuite> [testCase]  
2 [priority]
```

Beispiel:

```
1 startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml  
2
```

Einzelnen TestCase ausführen

```
1 startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml  
2 AGATE_Demo
```

Template instanziiieren

```
1 startTests.bat instantiate <appName> <templateFile> <dataFile>  
2
```

Beispiel:

```
1 startTests.bat instantiate TOSCATOAGATE AGATE_Demo.yaml  
2 TCD_AGATE_Demo.csv
```

YAML validieren

```
1 startTests.bat validate <yamlFile>
2
```

Beispiel:

```
1 startTests.bat validate data\TOSCATOAGATE\Instance_AGATE_Demo.yaml
2
```

DSL-Typen anzeigen

```
1 startTests.bat describe
2
```

oder:

```
1 startTests.bat describe REST
2 startTests.bat describe SOAP
3 startTests.bat describe CALL
4 startTests.bat describe SQL
5
```

9. Merksatz

```
1 describe
2   ↓
3 Wie sieht die AGATE DSL aus?
4
5 instantiate
6   ↓
7 Template + CSV -> konkrete Tests
8
9 validate
10  ↓
11 Ist das YAML technisch korrekt?
12
13 startTests
14   ↓
15 Test wirklich ausführen
16
```

Für Template-basierte Tests ist die wichtigste Reihenfolge:

```
1 INSTANTIATE -> VALIDATE -> EXECUTE
2
```

instantiate erzeugt aus einer wiederverwendbaren Testlogik und den dazugehörigen Testdaten eine konkrete Test Suite.

Erst diese konkrete Test Suite wird anschließend mit dem normalen AGATE Test Runner ausgeführt.